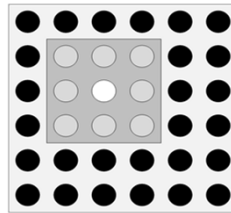




Universidade do Minho
Escola de Engenharia
Departamento de Produção e
Sistemas

Investigação Operacional e Optimização

Heurísticas



Filipe Alvelos

Universidade do Minho 2020

falvelos@dps.uminho.pt

Índice

- Optimização combinatória
- Heurísticas construtivas
 - mochila
 - caixeiro viajante
 - localização
 - afectação generalizada
- Heurísticas de pesquisa local
 - caixeiro viajante
 - afectação generalizada
- Notas finais

Optimização combinatória

$$\begin{aligned} & \text{Min } f(s) \\ & \text{sujeito a:} \\ & s \in S \end{aligned}$$

- s representa um objecto (solução) de um conjunto de objectos finito (ou infinito mas contável) representado por S (conjunto das soluções admissíveis)
- $f(s)$ é uma função de avaliação (objectivo ou custo) que faz corresponder a uma solução s um valor real — pode ser não linear ou mesmo um procedimento computacional
- Quando $f(s)$ é linear e $s \in S$ pode ser representado através de um conjunto de restrições lineares, existe (pelo menos) um modelo de Programação (linear) Inteira alternativo ao(s) modelo(s) de Optimização Combinatória

FA, IOO

3

Optimização combinatória

- Enumeração completa?
 - A enumeração completa das soluções admissíveis é implementável mas impraticável para instâncias de alguma dimensão
 - Para o problema do caixeiro viajante (assimétrico)
 - Dimensão do espaço de soluções (número de soluções) para uma instância com n cidades é $n!$
 - Para $n=10$, número de soluções: $10! = 3628800$
 - Para $n=30$, número de soluções : $30! = 2.65 \times 10^{32}$
 - Avaliando um bilião de soluções por segundo (um computador muito bom!), o tempo total seria mais de oito milénios!
 - Estima-se que a idade do universo seja 4.4×10^{17} segundos
 - Para $n=100$, número de soluções: $100! = 9.3 \times 10^{157}$
 - Estima-se que o número de átomos no universo esteja entre 10^{78} e 10^{82}

FA, IOO

4

Optimização combinatória

- Programação inteira?
 - Programação inteira permite a modelação de muitos problemas de optimização combinatória
 - Na prática, dada a interente complexidade do problema geral de programação inteira (i.e. NP-difícil) muitos problemas não podem ser resolvidos em tempo útil (e.g. árvore de pesquisa do método de partição e avaliação pode crescer exponencialmente)
 - Programação inteira pode servir de base a métodos aproximados em que a solução obtida não é necessariamente óptima (e.g., método de partição e avaliação truncado – método pára ao fim de algum tempo e retorna incumbente)

FA, IOO

5

Optimização combinatória

- Heurísticas e meta-heurísticas
 - Uma (meta-)heurística é um procedimento (computacional) que visa a obtenção de soluções satisfatórias (por contraste a soluções óptimas) num intervalo de tempo razoável
 - Uma heurística baseia-se em componentes específicos de um problema
 - Uma meta-heurística aplica princípios gerais utilizando esses componentes
- Características das heurísticas e meta-heurísticas
 - Flexibilidade
 - As funções envolvidas não precisam de ser lineares, nem sequer representáveis analiticamente
 - As funções envolvidas podem ir desde algumas linhas de código até “caixas pretas”
 - Utilizam conceitos simples e intuitivos
 - Permitem a exploração de conhecimento sobre o problema
 - São eficientes (mas nem sempre eficazes)
 - Podem ser concebidas muitas (meta-)heurísticas para um mesmo problema

FA, IOO

6

Heurísticas construtivas

- Ideia fundamental: construir uma solução, S , partindo da solução nula e adicionando sucessivamente elementos (obtendo assim soluções parciais intermédias) de acordo com uma regra pré-definida usualmente relacionada com a admissibilidade e o custo
- Princípios de uma heurística construtiva
 - Construção iterativa
 - Decisões tomadas são irreversíveis
 - Procedimento “guloso” (“greedy”)
 - Ter em conta os dados do problema mas ser independente dos dados da instância
- Em geral, as heurísticas construtivas podem não fornecer uma solução admissível
- A grande vantagem das heurísticas construtivas é a rapidez

FA, IOO

7

Heurísticas construtivas - mochila

- Problema da mochila
- Pretende-se seleccionar um subconjunto de itens, cada um caracterizado por um valor e um peso, de tal forma que o valor total (a soma dos valores dos itens seleccionados) seja o maior possível e o peso total (a soma dos pesos dos itens seleccionados) não ultrapasse um limite dado
- Exemplo. Considere-se um conjunto de encomendas que se pretende entregar a diferentes clientes existindo apenas um veículo disponível com uma capacidade de $6 m^3$. A cada encomenda estão associados o valor e o volume dados na Tabela. Pretende-se decidir as encomendas a entregar, já que não há capacidade suficiente para as entregar todas.

Encomenda	1	2	3	4	5	6	7	8	9
Valor (€)	123	134	125	112	140	124	133	122	150
Volume (m^3)	1.1	0.8	0.4	1.6	2.0	0.7	1.4	0.9	1.1

FA, IOO

8

Heurísticas construtivas - mochila

- Representação de uma solução
 - Conjunto de inteiros (cada inteiro é o índice de um item)
 - Exemplo $s^1 = \{1,4,5\}$
- **Heurística da razão** (para problema da mochila)
- Em cada iteração, incluir na mochila o objecto com maior razão entre o valor e o peso (que não exceda a capacidade). Parar quando não for possível inserir mais nenhum objecto ou quando se tiver tentado inserir todos os objectos
- Solução obtida pela heurística da razão no exemplo: $\{3,6,2,9,8,1\}$

FA, IOO

9

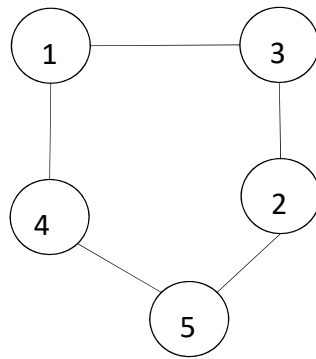
Heurísticas construtivas - PCV

- Problema do caixeiro viajante (PCV)
- O problema do caixeiro viajante pode ser representado numa rede não-orientada em que cada vértice corresponde a uma cidade e cada aresta $\{i, j\}$ corresponde às cidades i e j poderem ser visitadas uma imediatamente após a outra. A cada aresta $\{i, j\}$ está associada a distância entre i e j (no problema simétrico, o aqui tratado, a distância entre i e j é igual à distância entre j e i). Pretende-se determinar o circuito que passa por todos os vértices uma e uma só vez e que tenha menor custo total

	1	2	3	4	5
1	-	6	3	13	11
2	6	-	7	10	9
3	3	7	-	8	15
4	13	10	8	-	4
5	11	9	15	4	-

10

Heurísticas construtivas - PCV



	1	2	3	4	5
1	-	6	3	13	11
2	6	-	7	10	9
3	3	7	-	8	15
4	13	10	8	-	4
5	11	9	15	4	-

FA, IOO

11

Heurísticas construtivas - PCV

- **Heurística do vizinho mais próximo**
- Baseia-se na representação por permutação (elementos adicionados à solução são vértices)
- Considerar um nodo arbitrariamente e um caminho formado apenas por esse nodo
- Enquanto há nodos que não fazem parte do caminho
 - Determinar o nodo mais próximo em relação ao último considerado (de entre os que não fazem parte do caminho)
 - Incluí-lo no caminho
- Formar o circuito, adicionando ao caminho a aresta que liga o último nodo considerado ao primeiro
- Exemplo começando em 1: 1-3-2-5-4-1,
custo=3+7+9+4+13=36

	1	2	3	4	5
1	-	6	3	13	11
2	6	-	7	10	9
3	3	7	-	8	15
4	13	10	8	-	4
5	11	9	15	4	-

FA, IOO

Heurísticas construtivas - PCV

- Exemplo – Heurística do vizinho mais próximo (início em 1)

	1	2	3	4	5	6
1	100	2	6	4	15	11
2	2	100	4	19	12	2
3	6	4	100	7	8	9
4	4	19	7	100	17	4
5	15	12	8	17	100	1
6	11	2	9	4	1	100

	1	2	3	4	5	6
1	100	2	6	4	15	11
2	2	100	4	19	12	2
3	6	4	100	7	8	9
4	4	19	7	100	17	4
5	15	12	8	17	100	1
6	11	2	9	4	1	100

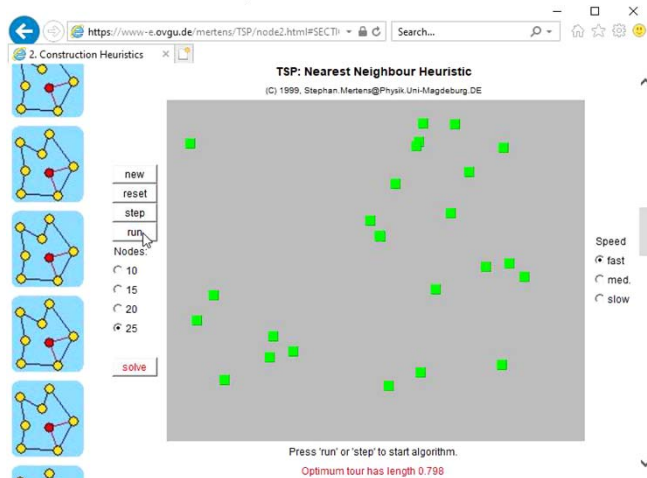
- Solução dada pela heurística do vizinho mais próximo: 1-2-6-5-3-4-1, valor 24

FA, IOO

13

Heurísticas construtivas - PCV

- Exemplo – Heurística do vizinho mais próximo (distâncias são as distâncias euclidianas)



FA, IOO

14

Heurísticas construtivas - PCV

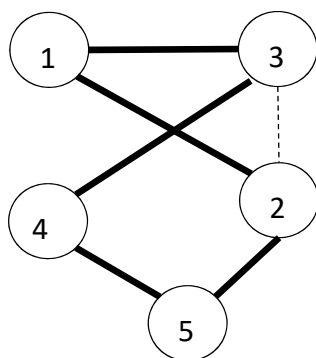
- **Heurística da aresta de menor custo**
- Baseia-se na representação por conjunto de arestas (elementos adicionados à solução são arestas)
- Uma aresta é não admissível se faz parte do conjunto de arestas já seleccionadas para o circuito
 - a sua selecção implica que um nodo fica com grau 3 (a cidade fica ligada a três cidades)
 - a sua selecção implicar a existência de um circuito com menos de n arestas
- Algoritmo
 - De todas as arestas admissíveis seleccionar aquela que tem menor custo
 - Repetir enquanto as arestas seleccionadas não formarem um circuito com n arestas

FA, IOO

15

Heurísticas construtivas - PCV

- Exemplo (por ordem de selecção das arestas):
- 1-3, 4-5, 1-2, (2-3 não é seleccionado por formar subcircuito), 3-4, 2-5
- Solução: 1-2-5-4-3-1, custo= 30



	1	2	3	4	5
1	-	6	3	13	11
2	6	-	7	10	9
3	3	7	-	8	15
4	13	10	8	-	4
5	11	9	15	4	-

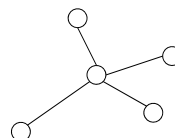
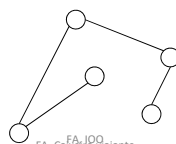
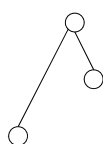
FA, IOO

16

Heurísticas construtivas - PCV

• Heurística da árvore de suporte de custo mínimo

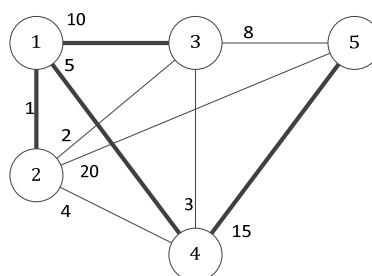
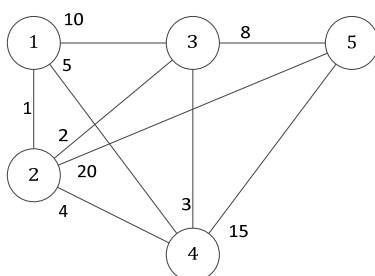
- Primeiro passo: obter árvore de suporte de custo mínimo (conjunto de arcos de baixo custo que não formam ciclos – embora vértices possam ter grau maior que 2)
- Segundo passo: transformar árvore de suporte em caixeiro viajante
- Um grafo não orientado com n vértices é uma árvore de suporte se tem $n-1$ arestas e não tem circuitos
- Uma árvore de suporte garante a conectividade da rede (existe um caminho entre qualquer par de vértices) com o menor número de arestas possível



17 17

Heurísticas construtivas - PCV

- Exemplo
- Do lado esquerdo, o problema, do lado direito uma solução formada pelas arestas $\{1,2\}$, $\{1,3\}$, $\{1,4\}$ e $\{4,5\}$ com custo 31.



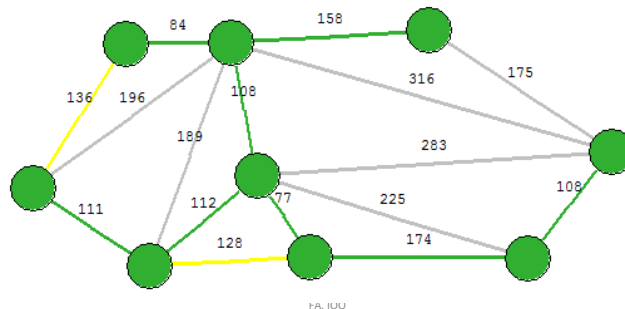
- Por exemplo, o conjunto de arestas $\{1,2\}$, $\{1,3\}$, $\{1,4\}$ e $\{3,4\}$ não é uma árvore porque, embora tenha $5 - 1 = 4$ arestas forma o circuito $(1,3,4,1)$

FA, IOO

18

Heurísticas construtivas - PCV

- Existem algoritmos eficientes para obter a árvore de suporte com menor custo como algoritmo de Kruskal
- Enquanto não estiverem seleccionadas $n-1$ arestas
 - Considerar a aresta de menor custo das que ainda não foram examinadas
 - Se a inserção dessa aresta não formar um ciclo então incluí-la na árvore de suporte
 - Se a inserção dessa aresta formar ciclo então excluí-la da árvore de suporte



FA, IUU

19

Heurísticas construtivas - PCV

- Ao contrário do costume, existem algoritmos construtivos para o problema da árvore de suporte de custo mínimo que obtêm a solução ótima!
- Por exemplo, o algoritmo de Kruskal
- Enquanto não estiverem seleccionadas $n-1$ arestas
 - Considerar a aresta de menor custo das que ainda não foram examinadas
 - Se a inserção dessa aresta não formar um circuito então incluí-la na árvore de suporte
 - Se a inserção dessa aresta formar um circuito, não a seleccionar e marcá-la como examinada
- No exemplo, a solução obtida (que é ótima) é $\{(1,2),(2,3),(3,4),(3,5)\}$ com custo 14. As arestas (2,4) e (1,5) foram examinadas mas formavam circuito

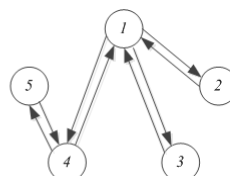
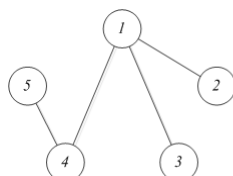
FA, IOO

20

Heurísticas construtivas - PCV

1. Obter árvore de suporte de custo mínimo
2. Duplicar todos os arcos da árvore de suporte de custo mínimo
3. Obter um circuito em que todos os arcos são percorridos uma e uma só vez (circuito Euleriano)
4. Transformar o circuito Euleriano num circuito em que todos os nodos são percorridos uma e uma só vez (circuito Hamiltoniano)

	2	3	4	5
1	6	3	7	11
2		7	10	9
3			8	7
4				4

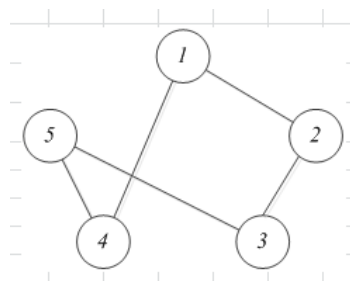
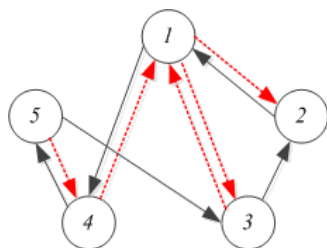


FA, 100

21

Heurísticas construtivas - PCV

- Circuito Euleriano 1-4-5-4-1-3-1-2-1
- Circuito Hamiltoniano 1-4-5-3-2-1



FA, 100

22

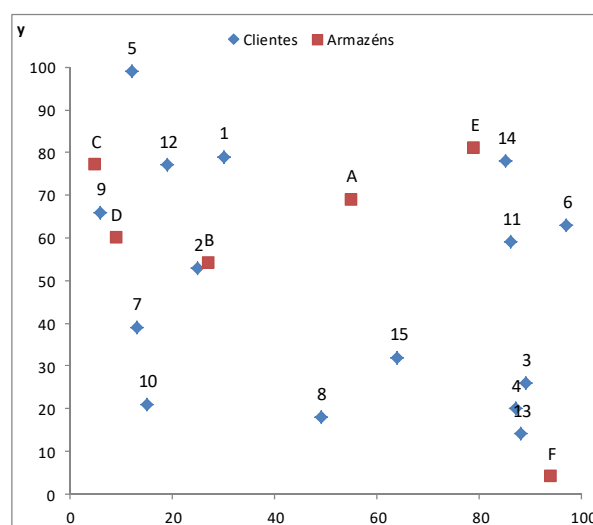
Heurísticas construtivas – localização

- Problema de localização
- Considere-se o problema de seleccionar a localização de um ou mais armazéns para servir um conjunto de clientes. As potenciais localizações do armazém (bem como o seu custo) e a localização dos clientes são dadas na Tabela e representadas no plano na Figura. O custo de servir um cliente a partir de um armazém é proporcional à distância entre ambos com a constante de proporcionalidade 5€/Km.

FA, 100

23

Heurísticas construtivas – localização



FA, 100

24

Heurísticas construtivas – localização

Cientes	X	Y	Potenciais armazéns	X	Y	Custo (€)
1	30	79	A	55	69	900
2	25	53	B	27	54	1200
3	89	26	C	5	77	1400
4	87	20	D	9	60	2100
5	12	99	E	79	81	800
6	97	63	F	94	4	400
7	13	39				
8	49	18				
9	6	66				
10	15	21				
11	86	59				
12	19	77				
13	88	14				
14	85	78				
15	64	32				

FA, IOO

25

Heurísticas construtivas – localização

- Representação de uma solução
 - Conjunto de inteiros (cada inteiro é o índice de uma localização)
 - Exemplo $s^1 = \{1,4,5\}$
 - Cálculo do custo não é directo
 - c_i custo de instalação do armazém i , c_{ij} custo de servir cliente j a partir do armazém i
 - $f(s) = \sum_{i \in s} c_i + \sum_{j=1}^n \min_{i \in s} c_{ij}$
- Heurística construtiva
 - Em cada iteração, a localização a abrir deve ser determinada pela maior variação de custo (negativa)
 - Parar quando todas as variações de custo forem positivas (ou nulas)

FA, IOO

26

Heurísticas construtivas – localização

- Abrir o armazém k , implica um custo adicional de c_k e uma poupança que corresponde à diferença entre o custo de servir os clientes a partir dos armazéns da solução actual e o custo de servir os clientes a partir dos armazéns da solução actual e do armazém k . Se esta diferença for positiva, não há alteração do custo, pelo que não é vantajoso abrir o armazém k . Formalmente, a variação do custo da solução por adição do elemento k é

$$\Delta_k = c_k - \sum_{j=1}^n \min \left\{ 0, \left(\min_{i \in x} c_{ij} - c_{kj} \right) \right\}$$

- No exemplo, a primeira localização adicionada é a A ($\Delta_A = 4333.3$) e depois a F ($\Delta_F = -250.1$). Depois todas as variações são positivas. A solução $(\{A, F\})$ tem valor 4083.2

FA, IOO

27

Heurísticas construtivas – AG

- Problema de afectação generalizada (AG)
- Na produção de um determinado artigo é necessário executar quatro tarefas. Para tal existem três máquinas, cada uma disponível durante 40 minutos. Dadas as diferentes características das máquinas e das tarefas a executar, o tempo de realização de cada tarefa depende da máquina em que é efectuada. Essa informação (expressa em minutos) é dada na primeira tabela. De acordo com a máquina utilizada, o custo de execução de cada tarefa foi estimado nos valores apresentados na segunda tabela. Cada tarefa tem de ser executada numa única máquina. Pretende-se a minimização do custo total.

FA, IOO

28

Heurísticas construtivas - AG

Tempos		Tarefas			
		1	2	3	4
Máquinas	1	10	15	20	25
	2	20	10	15	20
	3	15	15	20	20

Custos		Tarefas			
		1	2	3	4
Máquinas	1	100	80	60	40
	2	90	95	90	100
	3	80	70	70	50

FA, IOO

29

Heurísticas construtivas - AG

- Inicialização
 - Solução inicial = {}
 - Capacidades = [40, 40, 40]
- Primeira iteração
 - Solução = {(2,2)}
 - Capacidades = [40, 30, 40]
- Segunda iteração
 - Solução = {(2,2),(1,1)}
 - Capacidades = [30, 30, 40]
- Terceira iteração
 - Solução = {(2,2),(1,1),(1,4)}
 - Capacidades = [5, 30, 40]
- Quarta iteração
 - Par seleccionado (1,3) excede capacidade da máquina 1
 - Solução = {(2,2),(1,1),(1,4)}
 - Capacidades = [5, 30, 40]
- Quinta iteração
 - Solução = {(2,2),(1,1),(1,4), (2,3)}
 - Capacidades = [5, 15, 40]

$w_{ij} * c_{ij}$		Tarefas			
		1	2	3	4
Máquinas	1	1000	1200	1200	1000
	2	1800	950	1350	2000
	3	1200	1050	1400	1000

FA, IOO

30

Heurísticas construtivas - AG

- Seleccionar sucessivamente afectação com menor valor de $w_{ij} * c_{ij}$ de entre os possíveis (cada tarefa associada a um só agente e agente com capacidade limitada)
- Solução obtida
 - Agente 1 faz tarefas 1 e 4
 - Agente 2 faz tarefas 2 e 3
 - Agente 3 não faz nenhuma tarefa

FA, IOO

31

Heurísticas de pesquisa local

- Perspectiva das heurísticas de pesquisa local não é a de construir uma solução mas a de modificar uma solução existente para (potencialmente) obter uma solução melhor
- Em geral, não há forma óbvia de identificar as modificações que melhoram a solução, mas se forem em número reduzido, pode-se experimentar todas
- Algoritmo de pesquisa local
 - Obter uma solução actual (possivelmente com base numa heurística construtiva)
 - Obter a melhor solução vizinha da solução actual
 - Se a melhor vizinha é melhor do que a solução actual, então a melhor vizinha passa a ser a solução actual, ir para o passo anterior

FA, IOO

32

Heurísticas de pesquisa local - mochila

- **Estrutura de vizinhança de troca e adição**
- Uma solução é vizinha de outra quando pode ser obtida por troca de dois itens (um que está fora, entra e um que está dentro, sai) ou por adição de um objecto (entra item que está fora)
- Exemplo
 - Consideram-se 5 itens com os pesos da tabela e peso limite 10.

j	1	2	3	4	5
v_j	15	8	7	14	10
w_j	5	4	2	8	4
Razão	<i>3</i>	<i>2</i>	<i>3.5</i>	<i>1.75</i>	<i>2.5</i>

- Aplicando a heurística da razão, obtém-se a solução $\{1,3\}$ com valor 22

FA, IOO

33

Heurísticas de pesquisa local - mochila

- As soluções vizinhas desta solução e seus valores são:

$\{1,2\}$	23
$\{1,4\}$	Não admissível (rejeitada)
$\{1,5\}$	25
$\{3,2\}$	15
$\{3,4\}$	21
$\{3,5\}$	17
$\{1,3,2\}$	Não admissível (rejeitada)
$\{1,3,4\}$	Não admissível (rejeitada)
$\{1,3,5\}$	Não admissível (rejeitada)

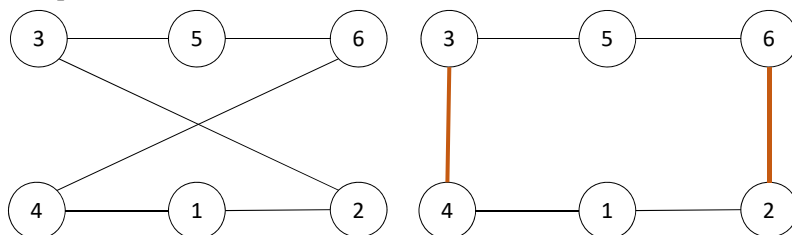
- A melhor vizinha é a solução $\{1,5\}$ com valor 25
- Como esta solução é melhor do que a actual, passa a ser a solução actual (caso contrário o algoritmo parava) e repete-se o procedimento (cálculo da vizinhança, escolha da melhor vizinha, decisão de actualizar ou parar)

FA, IOO

34

Heurísticas de pesquisa local - PCV

- Problema do caixeiro viajante
- Estrutura de vizinhança baseada em trocas de duas arestas (2-Opt)
- Exemplo solução actual 3-5-6-4-1-2-3



- Solução vizinha: 3-5-6-2-1-4-3
- A vizinhança de uma solução é o conjunto de soluções que se obtém ao remover duas arestas não adjacentes do circuito e a reconstruir o circuito com outras duas arestas
- Equivalente a inverter um caminho do circuito (um segmento da permutação): no exemplo, 3-5-6-**4-1-2**-3 passou a 3-5-6-**2-1-4**-3

FA, Caixeiro viajante

35

Heurísticas de pesquisa local - PCV

- Heurística de pesquisa local 2-opt
- Uma vizinha 2-opt pode ser obtida invertendo uma subsequência (de comprimento até de $n-2$ cidades) da permutação associada à solução

	1	2	3	4	5
1	-	6	3	13	11
2	6	-	7	10	9
3	3	7	-	8	15
4	13	10	8	-	4
5	11	9	15	4	-

- s é uma permutação
- Obter uma permutação, s^0 , por aplicação da heurística do vizinho mais próximo
 $s^0 = (1, 3, 2, 5, 4)$
- Obter e avaliar todas as soluções que se obtêm a partir de s^0 através de uma operação de troca de duas arestas $N(s^0) = \{t^{0,1}, t^{0,2}, t^{0,3}, t^{0,4}, t^{0,5}\}$

						Valor
s^0	1	3	2	5	4	36
$t^{0,1}$	1	2	3	5	4	45
$t^{0,2}$	1	5	2	3	4	48
$t^{0,3}$	1	3	5	2	4	50
$t^{0,4}$	1	3	4	5	2	30
$t^{0,5}$	1	3	2	4	5	35

FA, Caixeiro viajante

36

Heurísticas de pesquisa local - PCV

- Melhor solução obtida com troca de duas arestas é

$$t^{0,5} = (1,3,4,5,2)$$

- Nova iteração

$$s^1 = t^{0,5} = (1,3,4,5,2)$$

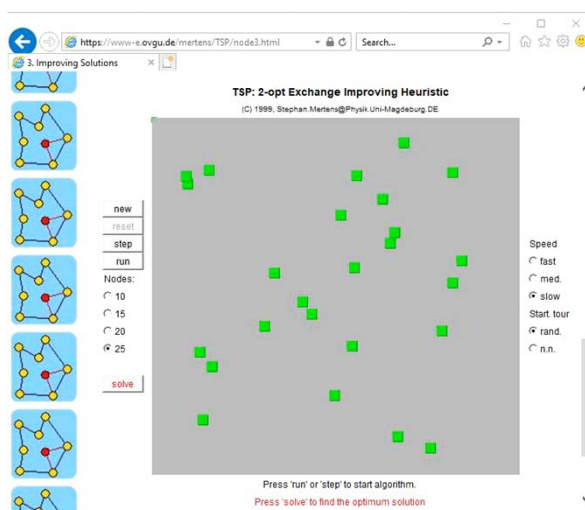
- Todas as soluções que se obtêm por troca de duas arestas são piores (ou iguais) do que a actual, logo a solução actual é uma solução óptima **local** (relativamente à estrutura de vizinhança de troca de duas arestas)

						Valor
s^1	1	3	4	5	2	30
$t^{1,1}$	1	4	3	5	2	51
$t^{1,2}$	1	5	4	3	2	36
$t^{1,3}$	1	3	5	4	2	38
$t^{1,4}$	1	3	2	5	4	36
$t^{1,5}$	1	3	4	2	5	41

FA, Calheiro viajante

37

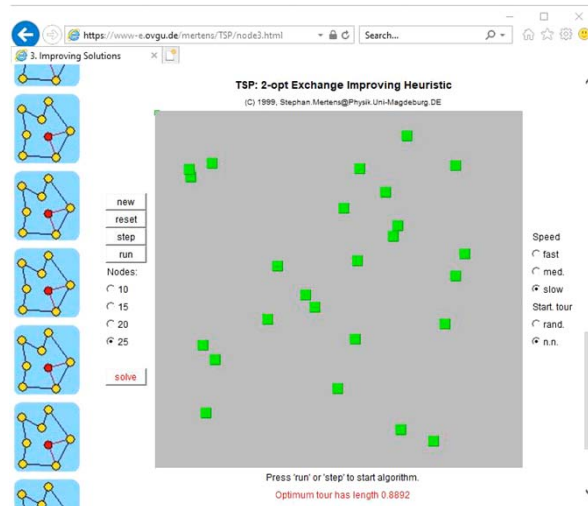
Heurísticas de pesquisa local - PCV



FA, Calheiro viajante

38

Heurísticas de pesquisa local - PCV

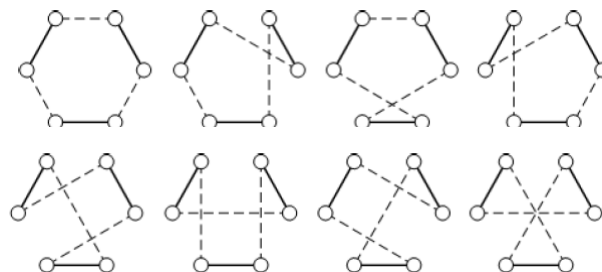


FA, Caixeiro viajante

39

Heurísticas de pesquisa local - PCV

- Estrutura de vizinhança baseada em **trocadas de três arestas (3-Opt)**
- A vizinhança de uma solução é o conjunto de soluções que se obtém ao remover três arestas não adjacentes do circuito e a reconstruir o circuito com outras três arestas (pelo menos uma diferente)
- 7 soluções vizinhas (4 soluções com três novas arestas e 3 soluções com duas novas arestas)

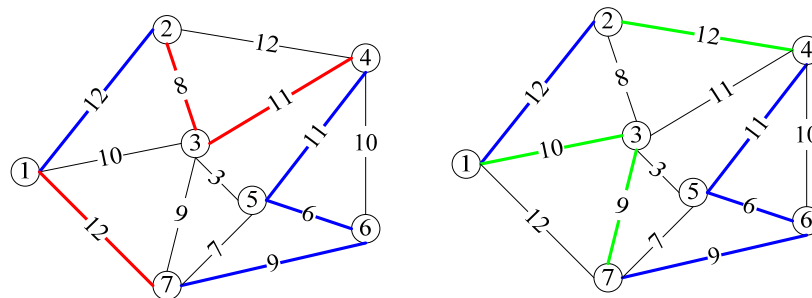


FA, Caixeiro viajante

40

Heurísticas de pesquisa local - PCV

- Estrutura de vizinhança baseada em **inserção de um vértice**
- A vizinhança de uma solução é o conjunto de soluções que se obtém ao remover um vértice do circuito e inseri-lo numa posição diferente
- Exemplo de uma solução e uma sua vizinha



FA, Caixeiro viajante

41

Heurísticas de pesquisa local - AG

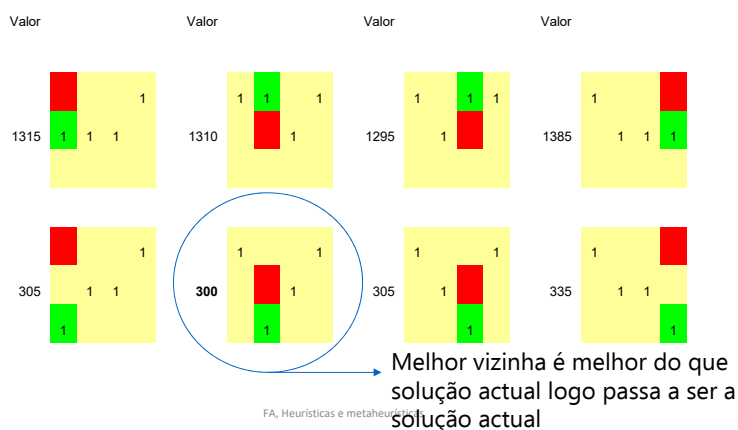
- Afectação generalizada (AG)
- Representação da solução
 - Três conjuntos de valores discretos (de 1 a 4) em que cada um aparece num e só num conjunto, $s = (s_1, s_2, s_3)$
 - Solução inicial: $s^0 = (\{1,4\}, \{2,3\}, \{\ \})$
 - Alternativamente, pode ser usada a representação de três vectores binários
- Estrutura de vizinhança
 - Uma solução é vizinha de outra se pode ser obtida pela afectação de uma tarefa a um agente diferente
 - Representação de conjuntos: mudar um valor de um conjunto para outro
 - Representação de vectores binários: mudar um valor 1 de um vector para outro
- Função objectivo: adiciona-se 1000 por cada restrição de capacidade violada

FA, Heurísticas e metaheurísticas

Heurísticas de pesquisa local - AG

- Primeira iteração
- Solução inicial (valor 325)
- Vizinhaça

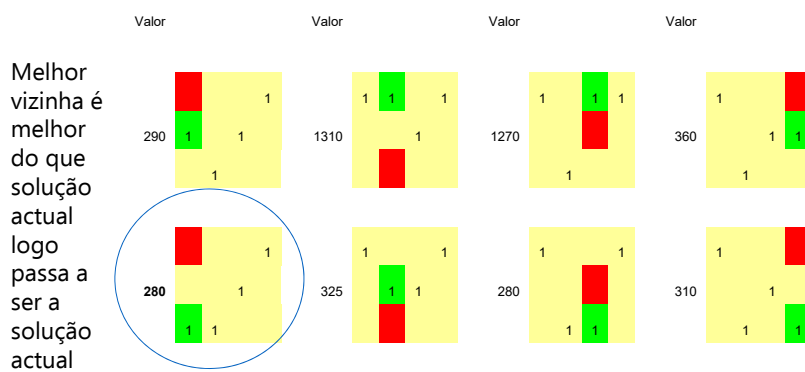
1			1
	1	1	



Heurísticas de pesquisa local - AG

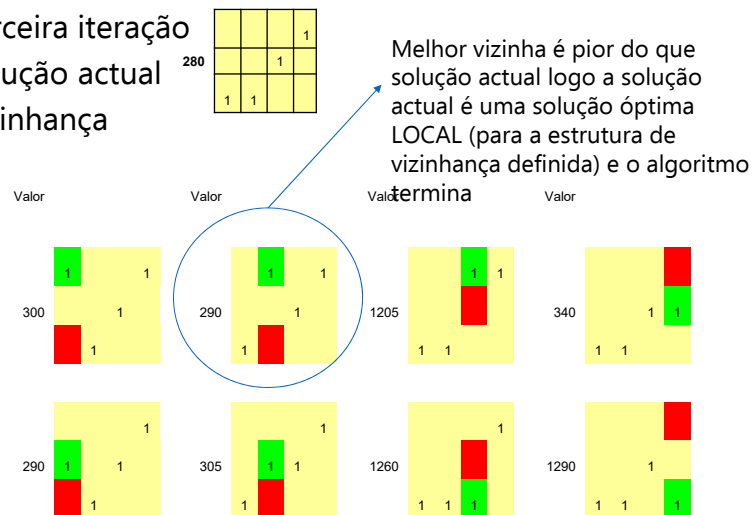
- Segunda iteração
- Solução actual ³⁰⁰
- Vizinhaça

1			1
		1	
	1		



Heurísticas de pesquisa local - AG

- Terceira iteração
- Solução actual
- Vizinhança



FA, Heurísticas e metaheurísticas

Notas finais

- Heurísticas
 - construtivas (simplicidade, esforço computacional baixo)
 - pesquisa local
 - meta-heurísticas (elaboradas, esforço computacional elevado, melhor qualidade das soluções) [não abordadas nestes apontamentos]
 - pesquisa tabu
 - algoritmos genéticos
 - ...

Exercício 6.1. Considere um problema em que se pretende seleccionar o conjunto de itens cuja soma do valor seja a maior possível sem ultrapassar o peso total de 20. As características de cada um dos seis itens são dadas na Tabela 6.1.

- Obtenha uma solução através da heurística construtiva da razão.
- Partindo da solução que obteve na alínea anterior, aplique uma heurística de pesquisa local baseada na estrutura de vizinhança de troca e adição.
- Descreva uma estrutura de vizinhança que permitisse a uma heurística de pesquisa local partir da solução da alínea anterior e obter a solução óptima, sabendo que esta é o conjunto dos itens 2, 3 e 4.

Item	1	2	3	4	5	6
Valor	3	6	14	5	4	9
Peso	1	4	12	4	4	10

Tabela 6.1: Dados do Exercício 6.1.

FA, 100

47

Exercício 6.2. Considere um problema do caixeiro viajante com 10 vértices definido no plano. As coordenadas de cada vértice são dadas na Tabela 6.2. As distâncias consideradas são as Euclidianas.

Vértice	x	y
1	40	19
2	19	83
3	75	75
4	16	44
5	89	76
6	76	79
7	2	2
8	100	87
9	86	41
10	14	94

Tabela 6.2: Dados do Exercício 6.2.

Tirando proveito da representação da Figura 6.1:

- Aplique a heurística do vizinho mais próximo partindo do vértice 7.
- Aplique a heurística da aresta de menor custo.

FA, 100

48

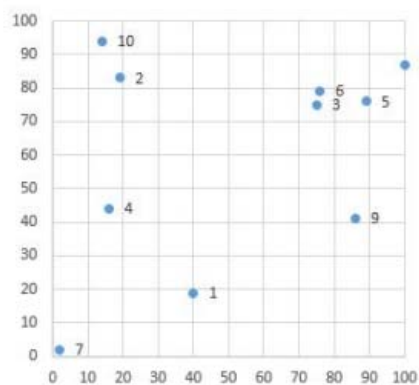


Figura 6.1: Representação no plano dos vértices do Exercício 6.2.

FA, 100

49

Exercício 6.3. Considere a instância do problema do caixeiro viajante da Figura 6.2.

- Aplique a heurística do vizinho mais próximo partindo do vértice 1.
- Aplique a heurística da aresta de menor custo.
- Existe a garantia de que alguma das soluções obtidas é ótima?

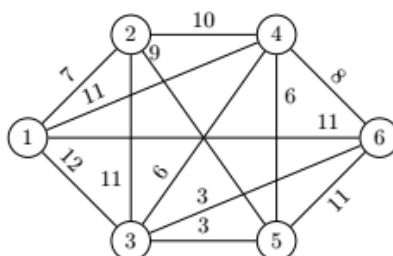


Figura 6.2: Rede do Exercício 6.3.

FA, 100

50

Exercício 6.4. Considere a instância do problema do caixeiro viajante com cinco vértices e distâncias dadas na Tabela 6.3.

- Aplice a heurística do vizinho mais próximo partindo do vértice 1.
- Aplice a heurística do vizinho mais próximo partindo do vértice 3.
- Mostre que as duas soluções obtidas nas duas alíneas anteriores são vizinhas na estrutura de vizinhança de troca de duas arestas.
- Aplice a heurística da aresta de menor custo.
- Mostre que a solução que obteve na alínea anterior é uma solução ótima local para a estrutura de vizinhança de duas trocas.

	1	2	3	4	5
1	-	9	10	18	12
2	9	-	6	5	19
3	10	6	-	20	7
4	18	5	20	-	17
5	12	19	7	17	-

Tabela 6.3: Dados do Exercício 6.4.

FA, 100

51

Exercício 6.5. Considere a instância do problema do caixeiro viajante com cinco vértices e distâncias dadas na Tabela 6.4.

	1	2	3	4	5
1	-	11	16	8	20
2	11	-	14	4	9
3	16	14	-	3	19
4	8	4	3	-	5
5	20	9	19	5	-

Tabela 6.4: Dados do Exercício 6.5.

- Aplice a heurística da aresta de menor custo.
- Partindo da solução da alínea anterior, obtenha uma solução ótima local para a estrutura de vizinhança de duas trocas.

FA, 100

52

Exercício 6.6. Uma determinada empresa de distribuição pretende definir onde localizar os seus três armazéns a partir dos quais serão abastecidos os seus dez principais clientes (C1 a C10). Para tal, seleccionou sete potenciais localizações (L1 a L7). Na Tabela 6.5 são dados os custos de cada cliente ser abastecido a partir de cada armazém.

	1	2	3	4	5	6	7	8	9	10
L1	395	182	325	142	450	294	491	215	119	133
L2	422	447	272	217	191	298	291	271	357	431
L3	339	465	195	200	274	140	486	221	243	176
L4	353	334	283	321	202	122	342	411	500	100
L5	118	499	407	379	486	259	303	474	259	133
L6	158	299	150	466	320	336	392	142	472	117
L7	374	487	257	493	214	132	221	229	462	331

Tabela 6.5: Dados do Exercício 6.6.

- Obtenha uma solução através de uma heurística construtiva, indicando claramente o critério de adição de um elemento à solução e a solução obtida em cada iteração.
- Partindo da solução que obteve na alínea anterior, aplique uma heurística de pesquisa local baseada numa estrutura de vizinhança por si definida.